

VIT VELLORE

SCOPE (School of Computer Science Engineering)

SPEECH AND LANGUAGE PROCESSING

A Project Report

Submitted in partial fulfilment of the requirements under the

Theory of Speech and Language Processing

BCSE419L

HyperBot: A Voice-Enabled Chatbot for Supercar and Hypercar Information using Speech Recognition and Deep Learning

Submitted by

Kanav	- Design, Implementation & Deployment	23BAI0018

Under the guidance of

Prof. Swetha Vasudevan

10 October 2026

VIT Vellore

1. Introduction

1.1 Problem Statement

Supercar and hypercar information (specifications, prices, production numbers and purchasing channels) is spread across manufacturer press releases, dealer websites and secondary-market listings. A spoken conversational assistant is a natural way to ask such questions, but it combines two language-processing problems. First, a speech recognition front end must turn the user's voice into text, and general-purpose recognisers often mishear exotic model names ("Chiron" → "sheeran", "Huayra" → "wire a"). Second, a natural language understanding component must work out what the user is asking for (the intent) and which car they mean (the entities), however the question is phrased. This project builds an end-to-end, publicly deployed voice chatbot that solves both problems. It uses a speech recognition pathway, entity extraction that is robust to recognition errors, and a deep-learning intent classifier, together with a dialogue manager that generates grounded answers from a curated knowledge base.

1.2 Objectives

1. Accept voice input and convert speech into text using a suitable speech recognition technique, with an on-device fallback when the browser's cloud speech service is unavailable.
2. Build a deep-learning intent classifier (Bidirectional LSTM with pre-trained GloVe embeddings) that maps a user utterance to one of 26 chatbot intents.
3. Make car-name recognition robust to speech recognition errors through n-best rescoring, fuzzy matching and phonetic matching.
4. Generate appropriate responses (details, specifications, price, where to buy, comparisons and recommendations) and display both the recognised speech and the chatbot response.
5. Evaluate the classifier honestly on phrasings never seen during training, against non-sequential and non-pretrained baselines.
6. Deploy the working application online with a publicly accessible link.

2. Literature Review

The following works were studied as the foundation for the speech recognition component, the word representations, the intent classification model, the dataset design, and the matching techniques used to correct recognition errors in this project.

2.1 Speech Recognition

[1] Hinton, G., Deng, L., Yu, D., Dahl, G. E., Mohamed, A., Jaitly, N., Senior, A., Vanhoucke, V., Nguyen, P., Sainath, T. N., & Kingsbury, B. (2012). *Deep Neural Networks for Acoustic Modeling in Speech Recognition: The Shared Views of Four Research Groups.* IEEE Signal Processing Magazine, 29(6), 82-97. This paper established deep neural acoustic models as the basis of modern commercial recognisers, including the cloud service behind the browser Web Speech API used as this project's default speech recognition engine.

[2] Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2022). *Robust Speech Recognition via Large-Scale Weak Supervision.* arXiv:2212.04356. This paper introduces Whisper, a transformer encoder-decoder trained on 680,000 hours of weakly supervised audio. Its *tiny.en* variant runs entirely in the browser as this project's fallback recogniser.

2.2 Word Representations

[3] Pennington, J., Socher, R., & Manning, C. D. (2014). *GloVe: Global Vectors for Word Representation.* Proceedings of EMNLP 2014, 1532-1543. This paper introduces the pre-trained word vectors used to initialise this project's embedding layer. With them, the classifier can interpret synonyms that never appear in the hand-written training data.

2.3 Recurrent Networks and Intent Classification

[4] Hochreiter, S., & Schmidhuber, J. (1997). *Long Short-Term Memory.* Neural Computation, 9(8), 1735-1780. This paper introduces the gated recurrent unit (the LSTM cell) at the core of the intent classifier, which models word order in variable-length utterances.

[5] Schuster, M., & Paliwal, K. K. (1997). *Bidirectional Recurrent Neural Networks.* IEEE Transactions on Signal Processing, 45(11), 2673-2681. This paper motivates reading each utterance in both directions, the bidirectional wrapper used in this project's model.

[6] Liu, B., & Lane, I. (2016). *Attention-Based Recurrent Neural Network Models for Joint Intent Detection and Slot Filling.* Proceedings of Interspeech 2016, 685-689. This paper demonstrates recurrent networks for spoken-language intent detection with slots, which informed this project's split into an intent classifier plus slot (entity) handling.

2.4 Dataset Design for Conversational Agents

[7] Hemphill, C. T., Godfrey, J. J., & Doddington, G. R. (1990). *The ATIS Spoken Language Systems Pilot Corpus.* Proceedings of the DARPA Speech and Natural Language Workshop, 96-101. This is the classic spoken-language-understanding corpus of domain-specific intents. No equivalent exists for the automotive domain, which motivated building a custom intent corpus.

[8] Larson, S., Mahendran, A., Peper, J. J., Clarke, C., Lee, A., Hill, P., Kummerfeld, J. K., Leach, K., Laurenzano, M. A., Tang, L., & Mars, J. (2019). *An Evaluation Dataset for Intent Classification and Out-of-Scope Prediction.* Proceedings of EMNLP-IJCNLP 2019, 1311-1316. This paper shows that deployed intent classifiers must handle off-topic queries, which motivated the explicit *out_of_scope* class and the confidence threshold used in this project.

[9] Wei, J., & Zou, K. (2019). *EDA: Easy Data Augmentation Techniques for Boosting Performance on Text Classification Tasks.* Proceedings of EMNLP-IJCNLP 2019, 6382-6388. This paper describes simple augmentations such as random word deletion. This project adapts them to simulate speech recognition errors in the training data.

2.5 Approximate String and Phonetic Matching

[10] Levenshtein, V. I. (1966). *Binary Codes Capable of Correcting Deletions, Insertions, and Reversals.* Soviet Physics Doklady, 10(8), 707-710. This paper defines the edit distance used to match

misrecognised car names to the database aliases.

[11] Philips, L. (1990). *Hanging on the Metaphone*. Computer Language, 7(12), 39-43. This paper introduces the Metaphone phonetic encoding. A simplified version lets this project match words that sound alike but are spelled differently ("sheeran" and "chiron" both encode to `xrn`).

3. Methodology

3.1 Dataset Description

Two datasets were built for this project. The first is an **intent corpus** of 796 hand-written utterance templates over 26 intents (19 to 54 templates per intent). It covers greetings and social turns, car-specific questions (overview, specifications, engine, power, top speed, acceleration, price, where to buy, production), comparisons, rankings, budget recommendations, definitions, and an *out_of_scope* class. Templates contain the slot placeholders `{car}`, `{brand}` and `{num}`, which are **delexicalised** into the tokens `xcar`, `xbrand` and `xnum`. The network therefore learns the intent of a sentence instead of memorising car names, and new cars can be added without retraining.

Each template is expanded into 12 variants. The variants add polite prefixes and suffixes, fill 12% of car slots with out-of-vocabulary tokens to simulate an unrecognised name, and apply random deletion of one non-entity word (probability 0.15) to simulate speech recognition errors. The real split sizes used are:

Split	Sentence Count
Random split: train	3,547
Random split: test	626
Template 5-fold CV: train (per fold, approx.)	3,239
Template 5-fold CV: test (per fold, approx.)	304
Final deployed model: all templates	4,158

In the template cross-validation, the held-out test sentences come from templates that are never used in training, and they are generated without word deletion. This measures generalisation to genuinely new phrasings rather than to re-worded copies of training sentences.

The second dataset is a **knowledge base** of 35 cars from 18 manufacturers (21 hypercars and 14 supercars; 2 fully electric and 17 hybrid). For each car it stores the engine, power, torque, 0-100 km/h time, top speed, approximate price in USD, production volume, availability status, a description, and aliases (including common mishearings). Specifications are manufacturer-claimed figures. Prices are approximate US base prices for new cars, or typical market values for sold-out and discontinued cars.

3.2 Speech Recognition

Voice input is captured in the browser. Two recognition engines are supported:

- **Web Speech API (default):** the browser vendor's neural recogniser. It streams interim results, which are shown live, and returns an n-best list of three alternatives with confidence scores. Supported accents are en-US, en-GB, en-IN and en-AU.

- **Whisper-tiny.en (fallback):** a 39M-parameter transformer encoder-decoder that runs on the device through Transformers.js and ONNX Runtime Web. Recorded audio is resampled to 16 kHz mono before transcription. The application switches to this engine automatically when the cloud speech service cannot be reached (for example in the Brave browser or on restricted networks).

Among the n-best alternatives, the top hypothesis is used unless only a lower-ranked one mentions a known car, in which case that alternative is chosen (n-best rescoring). The recognised text and its confidence are displayed next to every chatbot response.

3.3 Text Preprocessing and Entity Extraction

The recognised text is lower-cased and normalised: apostrophes are removed and punctuation is replaced, while decimals and model names such as "T.50" are preserved. Entities are then extracted in a fixed order:

1. exact car aliases, longest first (so that "Bugatti Chiron" wins over "Bugatti");
2. fuzzy car matches, where a window of one to three words is compared with every alias using normalised Levenshtein similarity (≥ 0.8) [10];
3. phonetic car matches, using a simplified Metaphone key [11];
4. brand matches.

Fuzzy and phonetic matching are applied only to windows that contain at least one uncommon word, to avoid false positives on everyday English. Monetary amounts and numbers are masked as `xnum`, while names such as "V12" are kept. A dedicated parser converts budgets to US dollars, including spoken forms ("two million", "half a million", "five hundred thousand") and Indian units ("3 crore"). Tokens are mapped to vocabulary indices (index 1 = out-of-vocabulary) and post-padded to a length of 24.

3.4 Intent Classification Model

The classifier is a Bidirectional LSTM [4, 5] implemented in TensorFlow/Keras:

- **Embedding layer:** 8,116 × 50, initialised with GloVe-50d vectors [3] (glove-wiki-gigaword-50) and fine-tuned, with padding masked. The vocabulary contains every training word plus the 8,000 most frequent English words. Rows for words absent from training receive no gradient and therefore keep their pre-trained meaning. The placeholder tokens start from the vectors of "supercar", "manufacturer" and "million".
- **Bidirectional LSTM:** 64 units per direction, giving a 128-dimensional sentence vector.
- **Classification head:** dropout 0.3 → dense 64 (ReLU) → dropout 0.3 → dense 26 (softmax).

The model has 474,626 parameters in total. It is trained with the Adam optimiser (learning rate 2×10^{-3}), sparse categorical cross-entropy, batch size 32 and a 10% validation split, for up to 80 epochs with early stopping on validation loss (patience 12, best weights restored). Two baselines are trained under identical conditions:

- a bag-of-words multilayer perceptron, which ignores word order;
- the same BiLSTM with embeddings learned from scratch, which uses no transfer learning.

For deployment, the trained weights are exported to a compact binary file, with int8-quantised embeddings and float32 elsewhere (≈0.7 MB). A hand-written JavaScript implementation of the masked

BiLSTM forward pass runs inference in the browser in a few milliseconds.

3.5 Dialogue Management and Response Generation

The dialogue manager combines the classifier output with the extracted entities:

- **Confidence threshold:** if the top intent probability is below 0.5, rule-based fallbacks use the entities (two cars → compare, one car → overview, a budget → recommendations). Otherwise the bot asks the user to rephrase.
- **Context memory:** the most recent cars are remembered, so that follow-ups such as "How fast is it?" and "Compare it with the Jesko" work. Context is reused only when a pronoun or a very short question indicates a follow-up. An unknown car ("Tesla Roadster") gets a clear "not in my database" answer instead of facts about the previous car.
- **Slot filling:** "Recommend me a car" is followed by "What is your budget?".
- **Where-to-buy answers** depend on availability:
  - in production: the official dealer and configurator;
  - limited production: the manufacturer, plus the secondary market;
  - sold out or discontinued: specialist marketplaces (duPont REGISTRY, JamesEdition) and auction houses (Bring a Trailer, RM Sotheby's).

Responses are shown as text with spec cards, comparison tables and ranked lists, and are read aloud with speech synthesis.

3.6 Deployment

The application is a static web application (HTML, CSS and JavaScript modules, the JSON knowledge base and the exported model weights). Speech recognition, entity extraction, inference and response generation all run in the browser, so no backend server or API key is needed. A GitHub Actions pipeline runs the automated test suite and then publishes the site to GitHub Pages.

4. Results

4.1 Intent Classification

Model	Random-split Accuracy	Random-split Macro-F1	Template CV Accuracy	Template CV Macro-F1
Bag-of-words MLP (baseline)	97.1%	0.930	80.6% ± 1.1	0.789 ± 0.028
BiLSTM, embeddings from scratch	94.9%	0.910	77.0% ± 1.3	0.740 ± 0.009
BiLSTM + GloVe (deployed)	**95.4%**	**0.918**	**81.5% ± 3.3**	**0.792 ± 0.044**

On the random split, test sentences can share a template with training sentences, and all three models exceed 94% accuracy. The bag-of-words model is slightly ahead there because keyword memorisation suffices. On the stricter template cross-validation, every test sentence is a phrasing the model has never

seen. There, pre-trained GloVe embeddings improve the BiLSTM by **4.5 accuracy points** over the same network trained from scratch, confirming the value of transfer learning. The GloVe-BiLSTM has the highest mean accuracy and macro-F1 of all three models, although its margin over the bag-of-words baseline lies within the fold-to-fold variation. The deployed model, trained on all templates, reaches **98.8% validation accuracy**.

4.2 Per-Intent Performance (Template CV, Fold 1)

Intent	Precision	Recall	F1-score	Support
greeting	1.00	0.62	0.77	8
goodbye	0.75	0.75	0.75	8
thanks	1.00	0.43	0.60	7
help	0.67	0.86	0.75	7
bot_identity	0.67	0.33	0.44	6
car_overview	0.59	0.79	0.68	24
car_specs	0.93	1.00	0.96	13
car_engine	1.00	0.87	0.93	15
car_power	1.00	1.00	1.00	11
car_top_speed	0.77	1.00	0.87	13
car_acceleration	0.80	0.67	0.73	12
car_price	0.94	0.88	0.91	17
where_to_buy	0.71	0.87	0.78	23
car_production	0.62	0.62	0.62	8
compare	0.90	0.94	0.92	18
fastest_car	1.00	1.00	1.00	11
quickest_car	0.64	0.90	0.75	10
most_powerful	1.00	1.00	1.00	10
most_expensive	0.88	1.00	0.93	7
cheapest	1.00	1.00	1.00	9
budget_recommend	1.00	0.85	0.92	13
electric_hybrid	0.91	0.83	0.87	12
list_cars	0.92	0.92	0.92	13
brand_info	0.92	0.92	0.92	12
definition	1.00	0.83	0.91	6
out_of_scope	1.00	0.36	0.53	11
Macro Average	**0.87**	**0.82**	**0.83**	**304**

The weakest intents are *bot_identity* (F1 = 0.44), *out_of_scope* (F1 = 0.53), *thanks* (F1 = 0.60). Their errors come from one-word slang the model has no embedding for ("thx", "cya"), off-topic questions

shaped like "who/what is X", and questions about the bot ("what technology do you use") that are confused with *help*. Intents tied to distinctive vocabulary (power, fastest, cheapest, most powerful) reach F1 = 1.00. In the running application, the dialogue manager limits the impact of misclassifications: a car-specific intent without any car mentioned produces a clarifying question rather than a wrong fact.

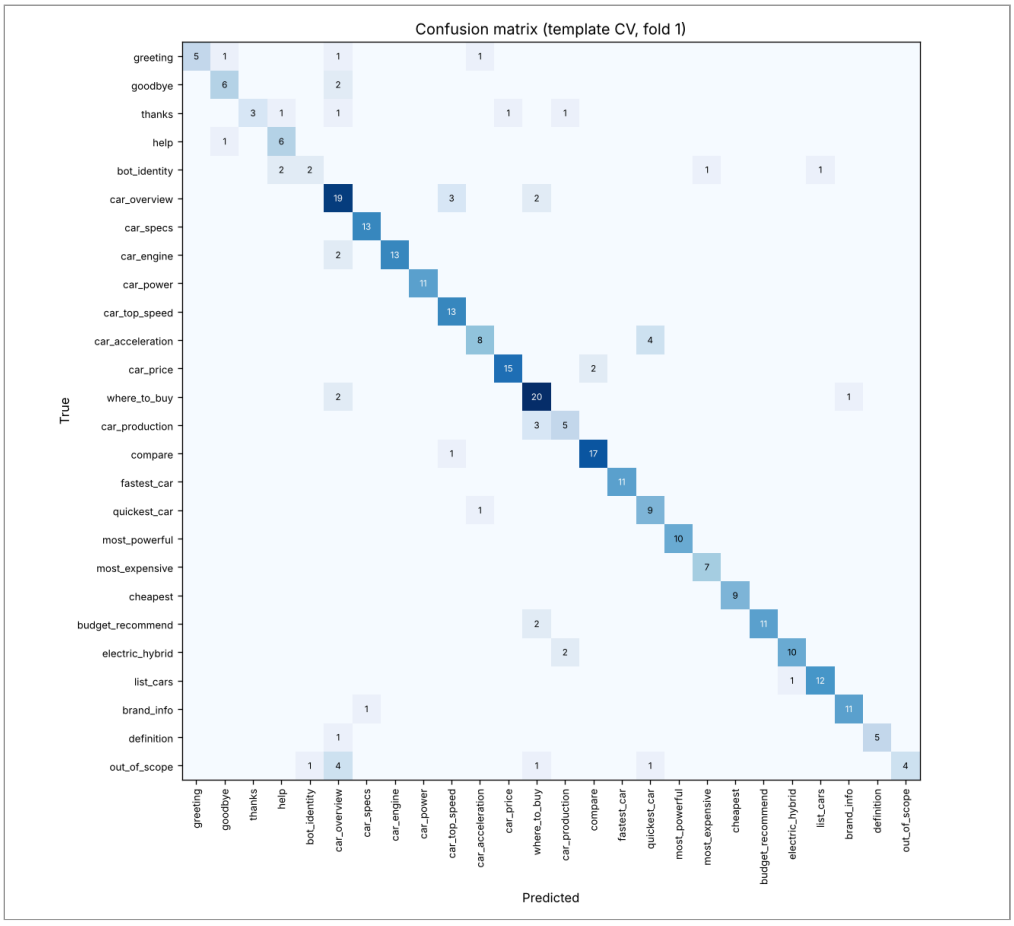


Figure 1: Confusion matrix of the deployed architecture on held-out templates (template CV, fold 1).

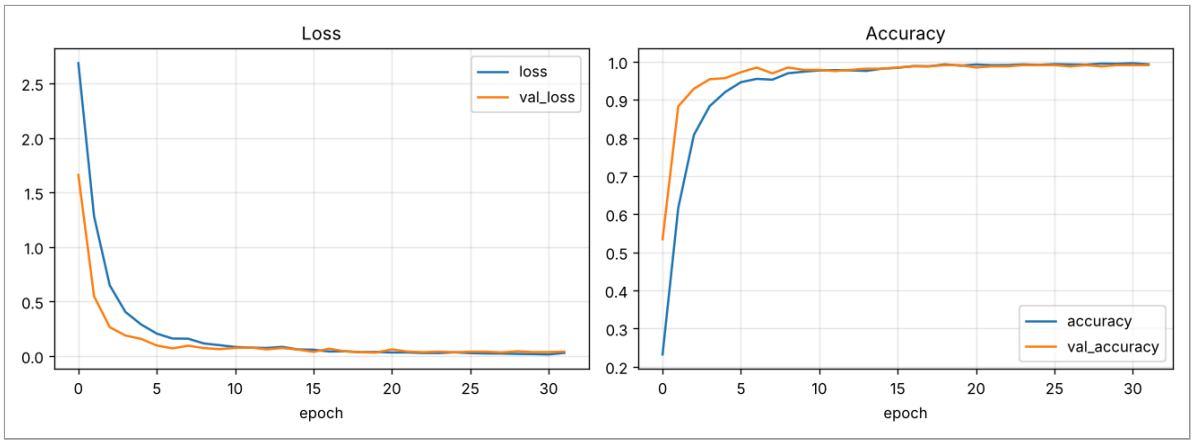


Figure 2: Training and validation loss and accuracy (fold 1); early stopping restores the best weights.

4.3 Speech-Error Robustness and System Tests

Test (run automatically before every deployment)	Result
Browser BiLSTM vs Keras reference predictions (11 sentences)	Identical labels; max probability difference 5×10^{-7}
Python vs JavaScript tokenizer (13 tricky inputs)	Identical
Entity extraction incl. misrecognised names ("bugatti sheeran", "koenig seg jesko", "rimac never a")	11 / 11 correct
Budget parser ("$300k", "2.5 million dollars", "two million", "half a million", "3 crore rupees")	8 / 8 correct
32-turn scripted conversation (follow-ups, comparisons, slot filling, out-of-scope)	32 / 32 intents and entities correct

4.4 Sample Dialogue

The table below is generated by running the deployed model on a scripted conversation:

User (recognised speech)	Predicted Intent	HyperBot Response
“Tell me about the Bugatti Chiron”	car_overview (100.0%)	The Bugatti Chiron Super Sport is a 2021 hypercar from Bugatti. The long-tail, top-speed focused Chiron. Its sibling was the first production car past 300 mph (490 km/h).
“How fast is it?”	car_top_speed (100.0%)	The Bugatti Chiron Super Sport has a top speed of 440 km/h (273 mph).
“How much does it cost?”	car_price (100.0%)	The Bugatti Chiron Super Sport costs about 3.9 million dollars.
“Where can I buy one?”	where_to_buy (100.0%)	The Bugatti Chiron Super Sport is sold out, so you'd have to buy one second-hand, through a specialist like duPont Registry, JamesEdition, or an auction house like RM Sotheby's. Bugatti dealers also broker resales.
“Compare it with the Koenigsegg Jesko”	compare (100.0%)	The Bugatti Chiron Super Sport wins on torque and 0–100 km/h, while the Koenigsegg Jesko Absolut wins on power, top speed and price.
“What can I buy with 400k?”	budget_recommend (100.0%)	With 400,000 dollars, my top pick is the Ferrari 296 GTB at about 340,000 dollars.
“bugatti sheeran price”	car_price (100.0%)	The Bugatti Chiron Super Sport costs about 3.9 million dollars.
“What is the price of the Tesla Roadster?”	unknown_car (100.0%)	Sorry, I don’t have that car in my database yet. Ask me what cars I know to hear the full list.

5. Demonstration

The system is delivered as a public web application. It shows the recognised speech with its recognition confidence, the chatbot response (with spec cards, comparison tables and purchase links), the predicted intent with its confidence, and a live panel that traces every stage of the language-processing pipeline. Answers are also spoken aloud. The screenshots below are captured from the running application.

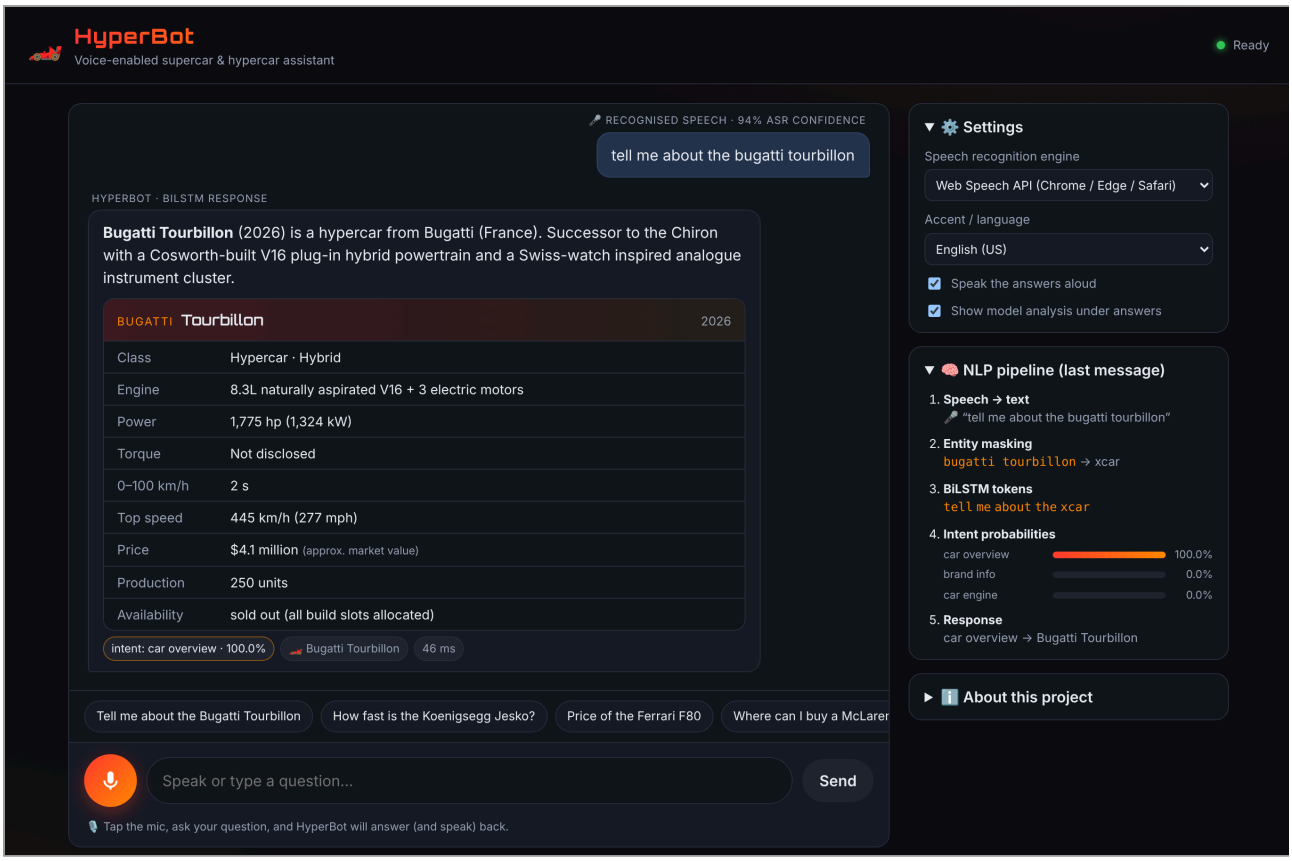


Figure 3: Voice query "tell me about the Bugatti Tourbillon": the recognised speech (94% confidence), the response with a spec card, the predicted intent and the NLP pipeline panel.

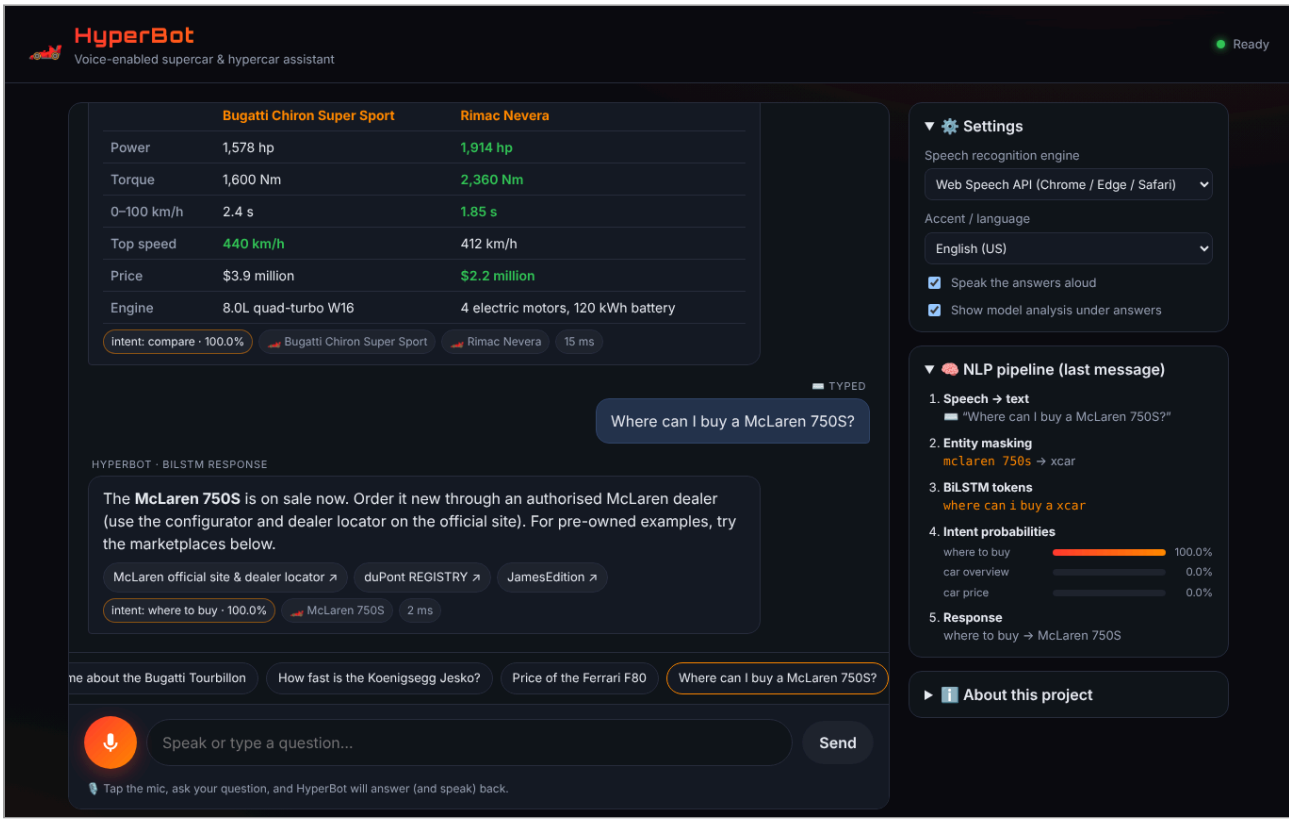


Figure 4: A comparison table (winners highlighted) and a where-to-buy answer with official-dealer and marketplace links.

5.1 Live Deployment, Repository and Local Setup

The application is deployed and publicly accessible at:

https://kanavbh29.github.io/slp/

The complete source code is hosted at:

https://github.com/KanavBh29/slp

```
git clone https://github.com/KanavBh29/slp
cd slp
pip install -r training/requirements.txt
python training/train.py                          # train, evaluate, export model
npm test                                          # run all tests
python -m http.server 8000 --directory web        # open http://localhost:8000
```

Speech input requires HTTPS (or localhost) and microphone permission. It works in Chrome, Edge and Safari, and other browsers fall back to the on-device Whisper engine.

6. Conclusion

This project implements a complete, publicly deployed voice-enabled chatbot for supercar and hypercar information. Speech is converted to text by a neural speech recognition engine, with an on-device Whisper fallback. A Bidirectional LSTM with transfer-learned GloVe embeddings classifies the user's intent among 26 classes, and a dialogue manager with context memory and slot filling generates grounded answers about the details, specifications, prices and purchasing options of 35 cars. On phrasings never seen during training, the deployed model reaches 81.5% accuracy, 4.5 points above the same architecture without pre-trained embeddings. Errors specific to speech recognition, such as misheard car names, are handled with n-best rescoring, fuzzy matching and phonetic matching. The interface displays both the recognised speech and the chatbot response, and the whole pipeline runs in the user's browser.

7. Future Work

- Replace the BiLSTM with a fine-tuned transformer encoder (e.g. DistilBERT) and compare accuracy against browser inference cost.
- Replace dictionary-based entity extraction with joint intent detection and slot tagging (e.g. a BiLSTM-CRF or the attention model of [6]).
- Collect real, anonymised user queries to extend the synthetic intent corpus, and measure end-to-end word error rate on recorded car-related speech.
- Connect the knowledge base to live listing data so that prices and availability stay current, and add multilingual speech input.

8. References

1. Hinton, G., Deng, L., Yu, D., Dahl, G. E., Mohamed, A., Jaitly, N., Senior, A., Vanhoucke, V., Nguyen, P., Sainath, T. N., & Kingsbury, B. (2012). Deep Neural Networks for Acoustic Modeling in Speech Recognition. *IEEE Signal Processing Magazine*, 29(6), 82-97.
2. Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2022). Robust Speech Recognition via Large-Scale Weak Supervision. *arXiv:2212.04356*.
3. Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global Vectors for Word Representation. *Proceedings of EMNLP 2014*, 1532-1543.
4. Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735-1780.
5. Schuster, M., & Paliwal, K. K. (1997). Bidirectional Recurrent Neural Networks. *IEEE Transactions on Signal Processing*, 45(11), 2673-2681.
6. Liu, B., & Lane, I. (2016). Attention-Based Recurrent Neural Network Models for Joint Intent Detection and Slot Filling. *Proceedings of Interspeech 2016*, 685-689.
7. Hemphill, C. T., Godfrey, J. J., & Doddington, G. R. (1990). The ATIS Spoken Language Systems Pilot Corpus. *Proceedings of the DARPA Speech and Natural Language Workshop*, 96-101.
8. Larson, S., et al. (2019). An Evaluation Dataset for Intent Classification and Out-of-Scope Prediction. *Proceedings of EMNLP-IJCNLP 2019*, 1311-1316.
9. Wei, J., & Zou, K. (2019). EDA: Easy Data Augmentation Techniques for Boosting Performance on Text Classification Tasks. *Proceedings of EMNLP-IJCNLP 2019*, 6382-6388.
10. Levenshtein, V. I. (1966). Binary Codes Capable of Correcting Deletions, Insertions, and Reversals. *Soviet Physics Doklady*, 10(8), 707-710.
11. Philips, L. (1990). Hanging on the Metaphone. *Computer Language*, 7(12), 39-43.